AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS MEDIANTE UNA API CON PERSISTENCIA Y VALIDACIÓN EN C#

202505232 - Angel Daniel Ruiz Ramos - Sección C

Resumen

Este ensayo examina el diseño de un servicio de autorización de documentos tributarios electrónicos y su relación con la integridad de la información. El problema consiste en recibir solicitudes JSON, verificar contribuyentes y cálculos monetarios, emitir códigos únicos y conservar resultados para consultas posteriores. Se sostiene que una respuesta correcta depende de coordinar validación, almacenamiento y comunicación, además de implementar las operaciones individuales. La solución separa una interfaz Razor Pages de una API desarrollada en C#, emplea listas enlazadas propias y utiliza archivos para mantener el registro tributario y el historial de solicitudes. Las expresiones regulares permiten extraer fechas sin rechazar el texto adicional que acompaña al mensaje. El análisis considera correlativos diarios, referencias duplicadas, concurrencia, reportes y recuperación ante errores de escritura. Desde una perspectiva técnica, la propuesta hace explícitas las decisiones que condicionan la interpretación de estadísticas y montos. Se concluye que la trazabilidad y la persistencia verificable son necesarias para que una autorización conserve su significado después de reiniciar el servicio.

Palabras clave

Autorización, persistencia, validación, trazabilidad, concurrencia.

Abstract

This essay examines the design of an electronic tax document authorization service and its relationship with information integrity. The problem involves receiving JSON requests, checking taxpayers and monetary calculations, issuing unique codes, and retaining results for subsequent queries. It argues that a correct response depends on coordinating validation, storage, and communication, as well as implementing individual operations. The solution separates a Razor Pages interface from an API developed in C#, uses custom linked lists, and stores the taxpayer registry and request history in files. Regular expressions extract dates without rejecting additional text accompanying the message. The analysis considers daily sequences, duplicate references, concurrency, reports, and recovery from write failures. From a technical perspective, the proposal makes explicit the decisions that affect the interpretation of statistics and monetary amounts. It concludes that traceability and verifiable persistence are necessary for an authorization to retain its meaning after the service restarts.

Keywords

Authorization, persistence, validation, traceability, concurrency.

Introducción

Autorizar un documento electrónico requiere relacionar reglas de negocio con mecanismos que preserven los resultados. El Proyecto 3 plantea un servicio HTTP que recibe mensajes JSON, verifica datos tributarios y produce consultas desde una aplicación web (Universidad de San Carlos de Guatemala, 2026b). Este ensayo analiza cómo distribuir esas responsabilidades mediante programación orientada a objetos en C#. Su propósito es explicar por qué la validación, la asignación de correlativos y la persistencia deben formar una operación coordinada. También examina la diferencia entre la fecha declarada en una factura y el momento de recepción, así como sus efectos en los reportes. La exposición parte de la arquitectura, desarrolla los algoritmos principales y concluye con sus alcances y límites.

Desarrollo del tema

Separación de responsabilidades

La solución utiliza dos procesos. El frontend contiene páginas Razor para seleccionar archivos, consultar información, descargar reportes y acceder a la documentación. El backend expone una API que puede recibir solicitudes de ese frontend o de un cliente externo. Esta separación permite comprobar las operaciones sin depender de controles visuales. El contrato HTTP constituye el punto de encuentro entre ambas aplicaciones y evita que la interfaz modifique directamente los archivos persistentes.

El envío de un archivo conserva los bytes recibidos. La aplicación web no agrega propiedades ni reconstruye el mensaje antes de transmitirlo. La API interpreta el arreglo JSON y devuelve los nombres de campos establecidos en el enunciado. Esta decisión mantiene la independencia del cliente: una solicitud enviada desde una herramienta HTTP tiene el mismo procesamiento que una solicitud enviada desde la página web. Los errores de estructura JSON se diferencian de los errores de negocio de cada factura.

Objetos y estructuras concretas

Contribuyente representa el NIT y el nombre del registro tributario. Factura conserva fecha, recepción, referencia, participantes, importes, autorización e indicadores de error. Estado reúne dos listas enlazadas concretas y ofrece la búsqueda de contribuyentes. Los nodos contienen una referencia al dato y otra al siguiente nodo. La separación permite recorrer el historial sin mezclar los enlaces con las propiedades tributarias. Las clases de dominio no requieren colecciones genéricas ni consultas LINQ.

ServicioTributario coordina las operaciones que modifican el estado. Repositorio carga y guarda el archivo; GeneradorReportes interpreta los datos para cada consulta. Las páginas reciben un ClienteApi, mientras GeneradorPdf transforma el reporte recibido en un documento. Estas responsabilidades se muestran en los diagramas de clases de los anexos. Compartir los tipos de reporte no significa compartir el estado: los dos programas siguen intercambiando información mediante HTTP.

Costo de los recorridos

Una lista enlazada facilita conservar registros e insertar al final. La búsqueda de un NIT requiere recorrer hasta c contribuyentes. Verificar referencias puede recorrer f solicitudes previas. Por ello, la autorización de lotes grandes puede implicar recorridos repetidos. Se acepta este costo para mantener una estructura cercana a los trabajos anteriores. No se afirma un tiempo logarítmico ni se presenta la implementación como una plataforma de procesamiento masivo.

Validación del mensaje

La extracción de fecha utiliza una expresión regular que identifica el segmento día, mes, año, hora y minuto dentro de TIEMPO. Después se comprueba que corresponda a una fecha real con horas entre cero y veintitrés. Nombres de lugares y sufijos como hrs. se ignoran. La expresión regular selecciona el contenido relevante; la conversión de calendario evita aceptar fechas imposibles. Si no existe una fecha válida, la solicitud queda rechazada y se contabiliza con su fecha de recepción.

El NIT se conserva como texto para no perder ceros iniciales y admitir hasta veinte dígitos anteriores al verificador. El cálculo comienza en el dígito situado a la izquierda del verificador, con peso dos, y aumenta hacia la izquierda. Se obtiene el módulo once de la suma y se aplica la regla indicada por el enunciado. Un resultado diez corresponde a K. La validación matemática precede a la búsqueda en el RTU; un NIT inválido y uno válido inexistente representan causas distintas.

Importes y referencias

Los importes se trabajan con decimal. El valor debe ser positivo y representable con dos decimales; IVA y TOTAL no pueden ser negativos. El IVA esperado se calcula multiplicando el valor por 0.12 y redondeando a dos posiciones. Se emplea redondeo de mitades alejándose de cero. El total esperado se obtiene sumando el valor y el IVA recibido. Ambas comprobaciones son independientes, por lo que una misma solicitud puede acumular más de un error.

La referencia distingue mayúsculas y minúsculas y admite hasta cuarenta posiciones. Se impide reutilizar una referencia registrada, incluso si la solicitud anterior fue rechazada, se recibió en otro día o precede a un reinicio. Una solicitud corregida debe utilizar una nueva referencia. Esta interpretación aplica la unicidad a todo el historial recibido, porque el enunciado no limita expresamente la regla a documentos aprobados. La referencia vacía produce un error de formato y no se considera una clave válida.

Autorización y fecha de recepción

El código concatena la fecha de recepción en Guatemala con un correlativo de ocho posiciones. Se consulta el mayor correlativo almacenado para ese día y se incrementa únicamente cuando la factura es aprobada. La fecha declarada en TIEMPO sirve para agrupar los movimientos en los reportes. Así, una factura fechada en enero y recibida en octubre conserva enero en las consultas y octubre en su autorización. Separar ambos campos evita que mensajes antiguos reinicien la secuencia actual.

Las respuestas de autorización conservan los cuatro campos del objeto resultadoAutorizaciones, incluida la lista detalleAprobaciones. Los siete tipos de errores solicitados se consultan en las estadísticas; no se agregan campos al contrato de respuesta obligatorio. Las cantidades de errores no equivalen necesariamente a la cantidad de facturas rechazadas, porque una factura puede incumplir varias reglas. El reporte también conserva un contador de formato incorrecto para explicar rechazos que no pertenecen a las siete categorías enumeradas.

Persistencia y coherencia

El archivo estado.json contiene el RTU y todas las solicitudes procesadas. El backend lo carga al iniciar. Para modificarlo, el servicio copia el estado, aplica la operación a esa copia, escribe un archivo temporal y reemplaza el archivo definitivo. Solo después publica la copia como estado vigente en memoria. Si la escritura falla, la respuesta no anuncia éxito y el estado anterior permanece disponible. Esta secuencia reduce la posibilidad de comunicar una autorización que solo exista en memoria.

Un bloqueo de acceso serializa las operaciones de la instancia. De esta manera, dos solicitudes concurrentes no leen el mismo correlativo y lo asignan simultáneamente. Un archivo de bloqueo exclusivo impide iniciar un segundo backend sobre la misma carpeta de datos. Esta protección corresponde a una aplicación de una instancia y no constituye un mecanismo distribuido. La escritura temporal y el reemplazo se realizan en la misma carpeta para conservar la operación dentro del mismo sistema de archivos.

Persistir las facturas permite reconstruir el correlativo máximo de cada día sin un contador separado que pueda quedar desincronizado. También preserva las referencias utilizadas y los motivos de rechazo. Cuando un archivo existente está corrupto, el servicio no lo sustituye silenciosamente por un estado vacío: el inicio falla y requiere revisar el almacenamiento. El borrado explícito ocurre únicamente mediante la operación de inicialización, que también guarda de forma persistente el estado vacío.

Consultas y representación

El reporte de contribuyente distingue facturas emitidas y recibidas, agrupa por fecha y calcula cantidades autorizadas y total facturado. El listado complementario muestra los documentos individuales con referencia, emisor, receptor, valor, IVA, total y código. El resumen de IVA acumula por contribuyente el impuesto emitido y recibido en una fecha. Un documento con el mismo NIT en ambos extremos participa en ambas columnas, porque representa dos perspectivas del mismo movimiento.

El resumen por rango ofrece total con IVA o valor sin IVA. Las gráficas utilizan barras cuya longitud corresponde al importe y cuyo valor numérico también aparece escrito. Los reportes PDF incluyen los filtros, los datos, el total y las gráficas correspondientes. La paginación evita que las consultas extensas queden cortadas. Las estadísticas reúnen todos los intentos, incluidos los rechazados; los reportes monetarios consideran únicamente facturas autorizadas.

La cantidad de emisores y receptores distintos en las estadísticas se obtiene sobre los identificadores no vacíos recibidos, incluso cuando presentan errores. Esa elección permite describir el contenido de las solicitudes, mientras las consultas monetarias describen documentos aprobados. Las fechas sin movimientos se muestran con cero dentro del rango. Exponer estas convenciones evita interpretar las cifras como si todos los reportes utilizaran el mismo universo de datos.

Verificación y alcance

La verificación combina comprobaciones del dominio con solicitudes HTTP. Las pruebas de dominio incluyen NIT, calendario, persistencia, cambio de día, referencias repetidas, concurrencia y fallos de escritura. Las pruebas HTTP comprueban contratos de respuesta, formatos incorrectos, filtros, páginas, gráficas y generación de PDF. También se inspecciona la presentación en navegador y los documentos renderizados. El registro de resultados se conserva por separado para distinguir observaciones ejecutadas de requisitos aún pendientes.

La aplicación corresponde a un simulador académico de autorización y no a una conexión con un servicio real de la SAT. Las fórmulas y reglas implementadas provienen del enunciado del proyecto; no constituyen asesoría tributaria. El almacenamiento por archivos y los recorridos secuenciales se diseñaron para una ejecución local. No se ha demostrado tolerancia a fallos de hardware, alta disponibilidad ni rendimiento de producción. La configuración de ejecución conserva separadas las carpetas de prueba y los datos del usuario.

Conclusiones

La separación entre Razor Pages y la API permite verificar el contrato de comunicación de manera independiente. Sin embargo, esa separación solo resulta útil cuando las reglas del dominio se concentran en el servicio y los clientes reciben resultados coherentes. Las listas enlazadas y las clases concretas ofrecen una implementación legible cuyo costo debe reconocerse: conservar claridad no equivale a garantizar escalabilidad para cualquier volumen de documentos.

La integridad de una autorización exige coordinar el correlativo con el almacenamiento. Un código único durante una ejecución pierde utilidad si vuelve a emitirse al reiniciar. Guardar el historial y publicar el estado después de completar la escritura mantiene esa relación. Los bloqueos complementan el procedimiento al impedir asignaciones concurrentes sobre la misma secuencia. Estas medidas se verifican dentro del alcance de una instancia local y no sustituyen una arquitectura distribuida.

Las estadísticas adquieren significado cuando se explican sus criterios. La fecha de la factura, el día de recepción, los intentos rechazados y los documentos aprobados describen aspectos distintos. Documentar las convenciones permite interpretar los reportes y discutir las ambigüedades del enunciado. Finalmente, la preparación técnica debe acompañarse de los requisitos externos de entrega: repositorio, releases, colaborador y envío. La existencia de archivos locales no demuestra el cumplimiento de esas condiciones académicas.

Referencias bibliográficas

Universidad de San Carlos de Guatemala. (2026a). Formato de artículo o ensayo del laboratorio de Introducción a la Programación y Computación 2 [Plantilla del curso].

Universidad de San Carlos de Guatemala. (2026b). Proyecto 3: Introducción a la Programación y Computación 2, segundo semestre de 2026, versión 1 [Enunciado del curso].

Anexo A. Modelo de dominio y persistencia

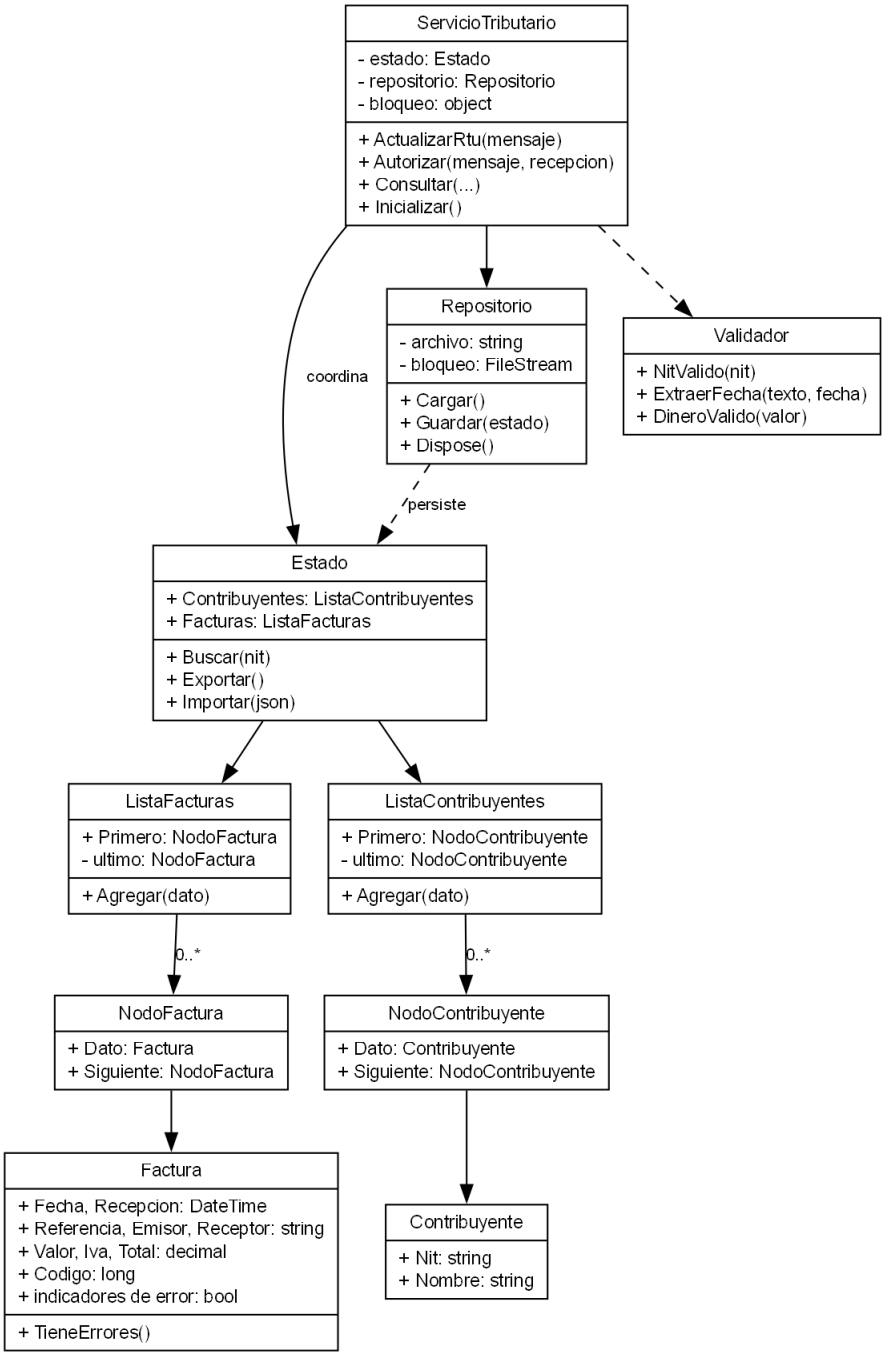


Figura 1. Modelo de dominio y persistencia. Fuente: elaboración para esta implementación.

Anexo B. Modelo de comunicación y reportes

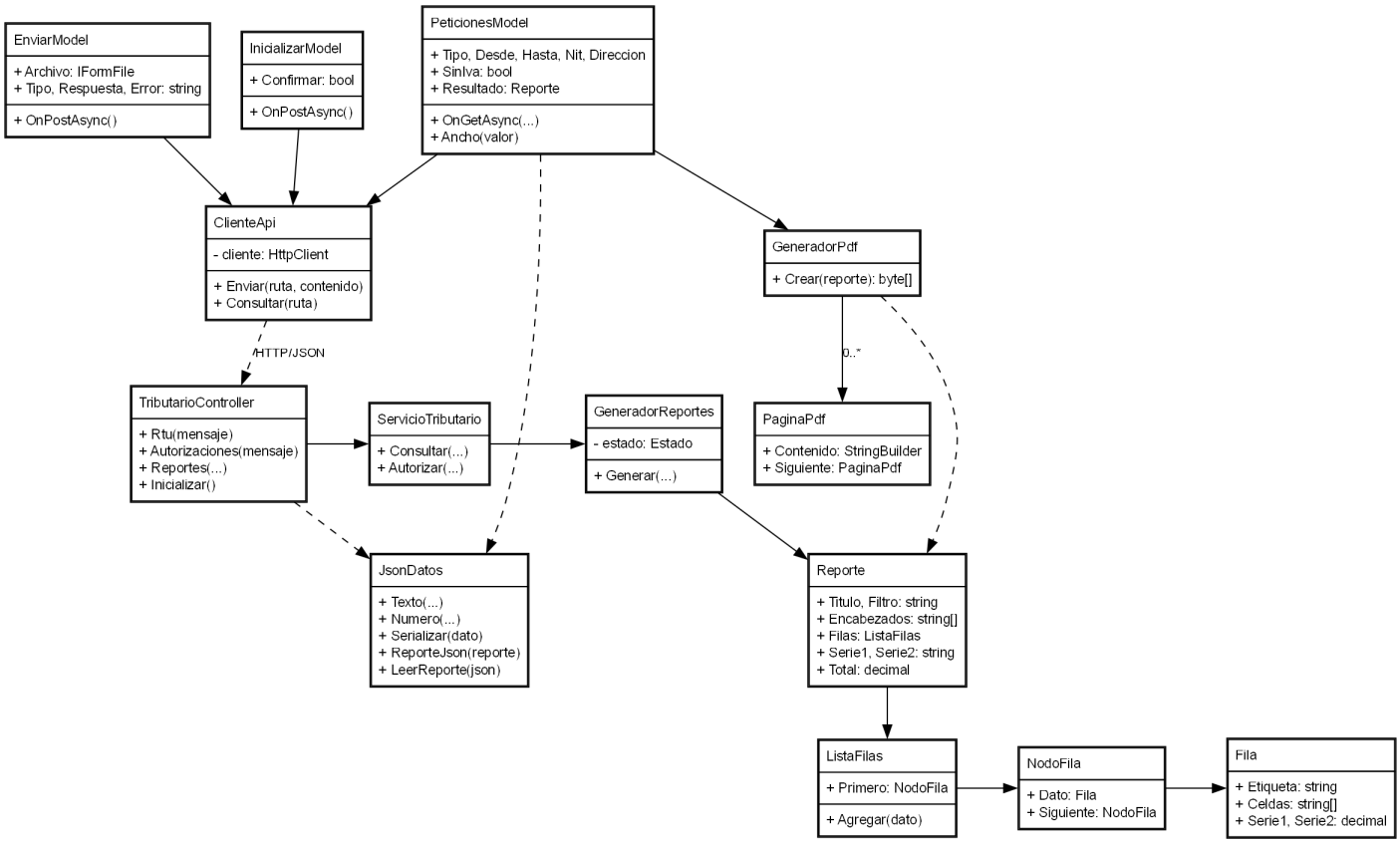


Figura 2. Modelo de comunicación y reportes. Fuente: elaboración para esta implementación.

Anexo C. Actividades de actualización y autorización

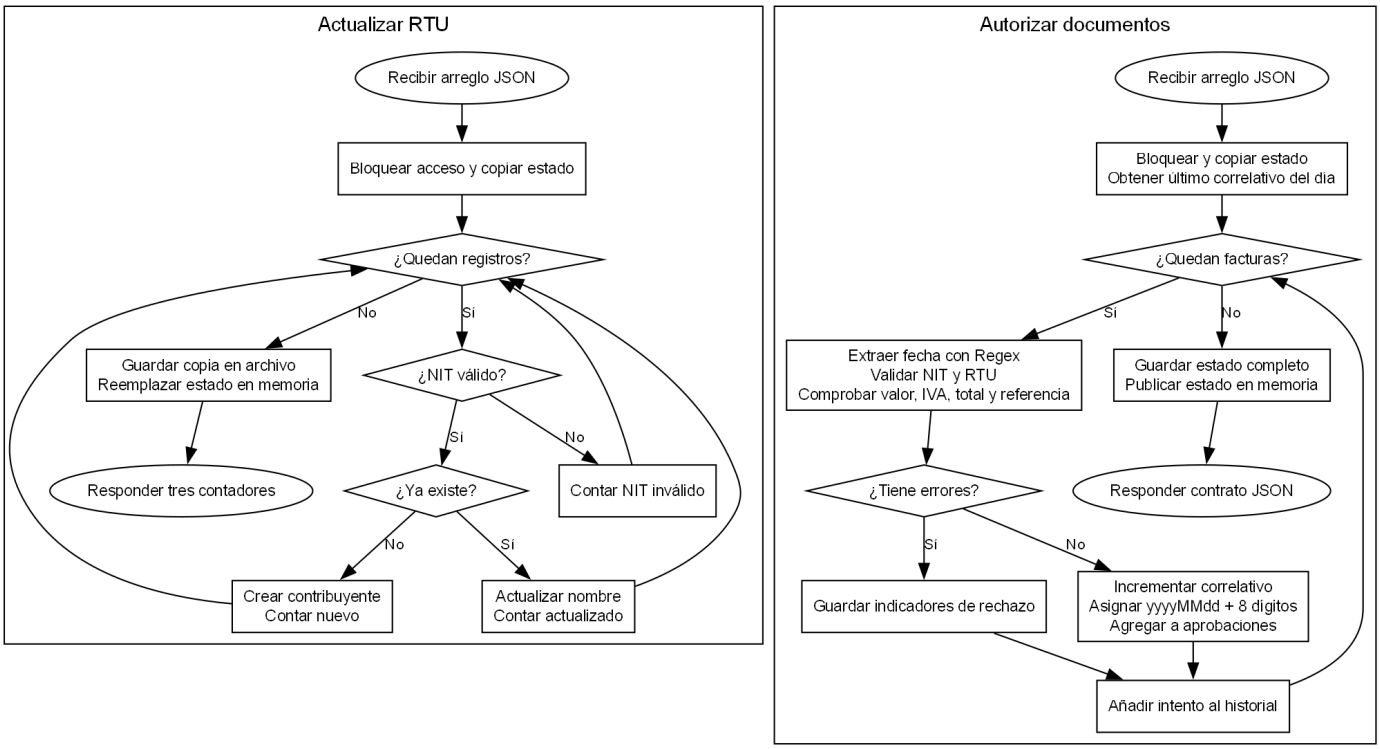


Figura 3. Actividades de actualización y autorización. Fuente: elaboración para esta implementación.

Anexo D. Actividad de consulta y exportación

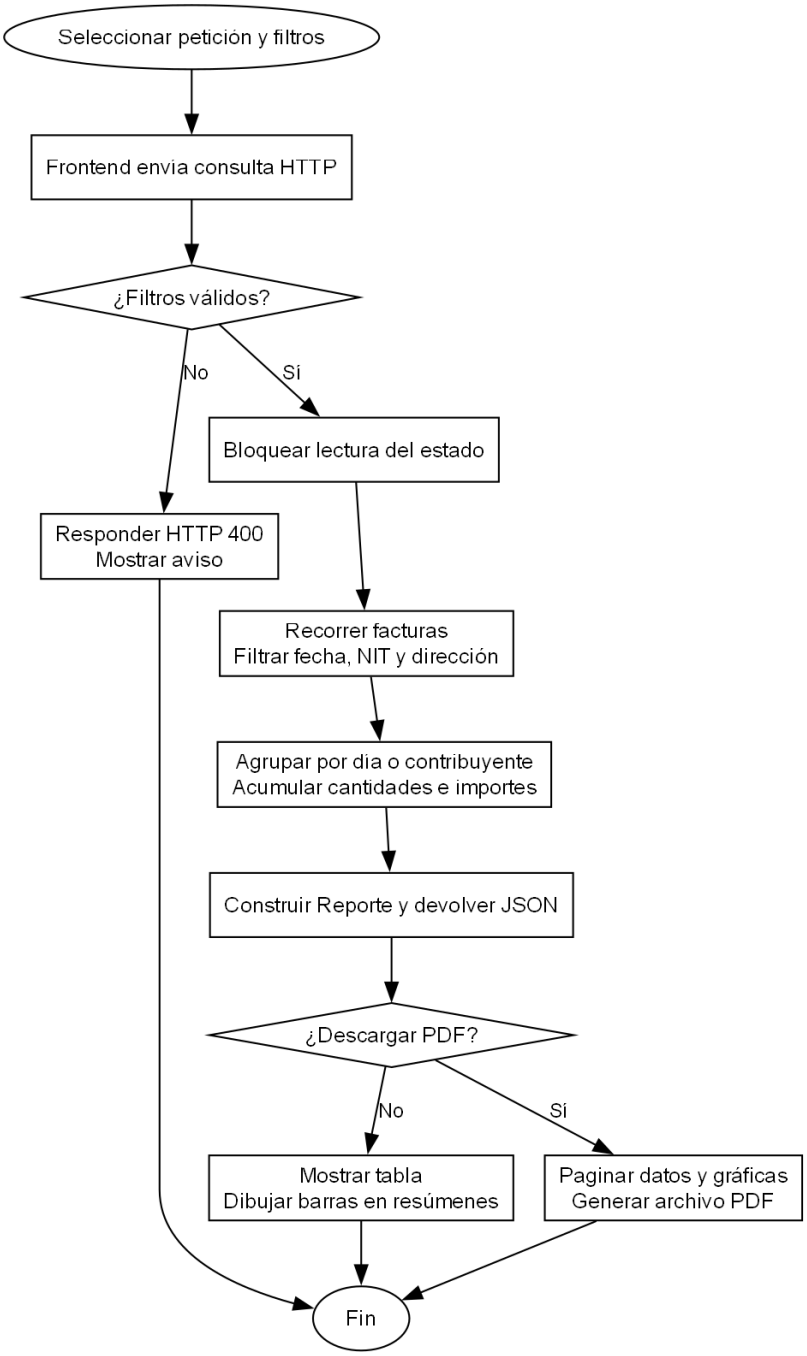


Figura 4. Actividad de consulta y exportación. Fuente: elaboración para esta implementación.

Anexo E. Despliegue del sistema

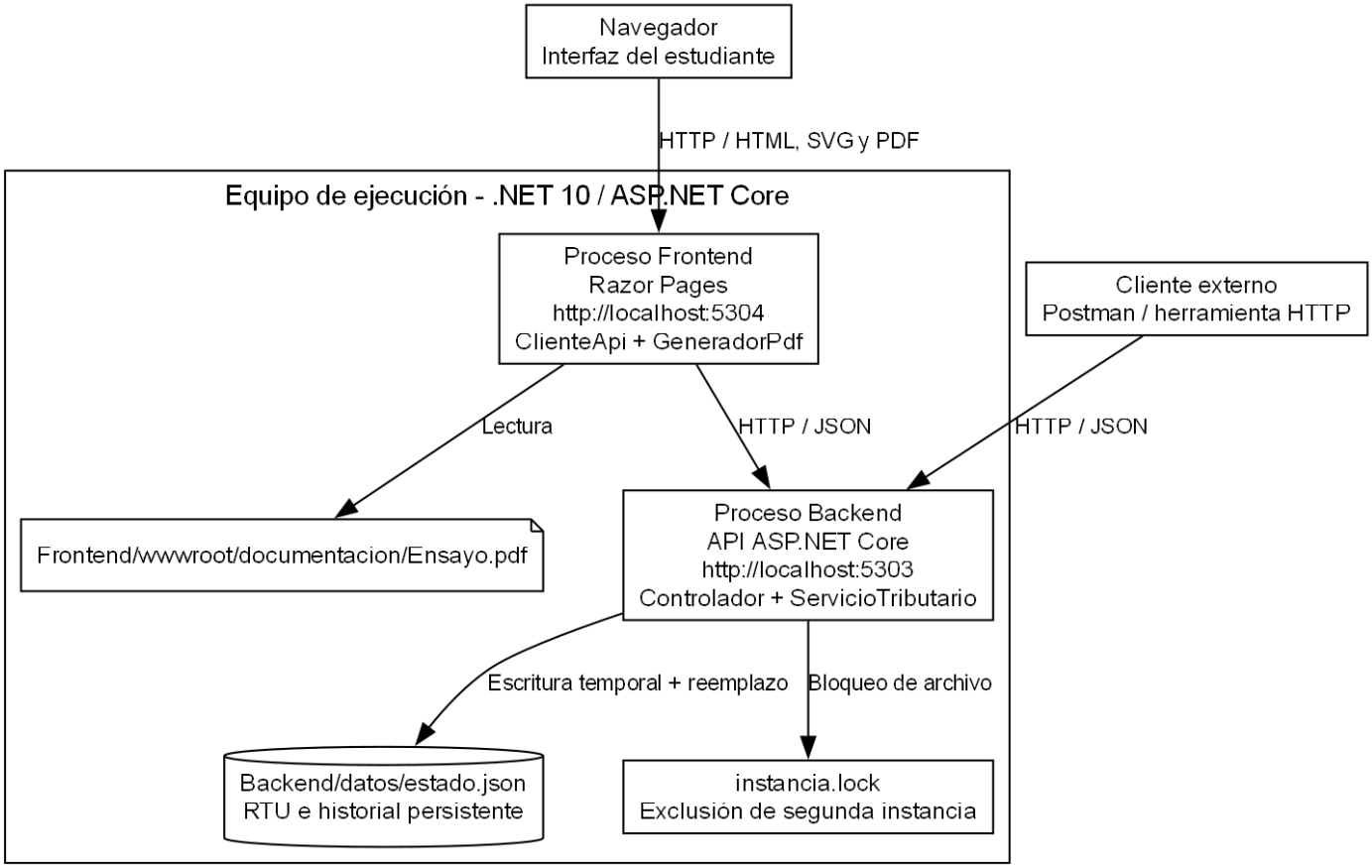


Figura 5. Despliegue del sistema. Fuente: elaboración para esta implementación.